SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1004	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:		**Reg. No.:**	
Date:	27.10.26	**Duration:**	60 minutes

Code of Conduct

*I **M.DEVI SRILATHA (192525236)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: ***Srilatha***

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:
Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description

- Scope of the System
- Functional Requirements
- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS)	SRS is well-structured, complete, professionally organized, and	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or

Documentation (15%)	includes all required sections.			lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25

1. Study and Analyze the Scenario

Business Domain

Legal Technology (LegalTech).

Problem Description

Law firms handle many legal documents, such as contracts, agreements, notices, and case files. Managing them manually takes time and makes searching, tracking versions, and remembering deadlines difficult.

Stakeholders:

1. Lawyers
2. Legal Assistants
3. Administrators
4. Clients

Existing Challenges:

- Difficult to find documents.
- Manual work takes more time.
- Confusion between document versions.
- Important deadlines may be missed.
- Risk of unauthorized access.

Objectives:

- Store documents securely.
- Classify documents using AI.
- Search documents easily.
- Maintain document versions.
- Track document access.
- Send deadline reminders.
- Organize case documents.

2. Functional and Non-Functional Requirements

A. Functional Requirements

ID	Requirement
FR1	User login and logout
FR2	Upload and store documents
FR3	Classify documents using AI
FR4	Search and retrieve documents
FR5	Maintain document versions
FR6	Manage legal cases
FR7	Record deadlines
FR8	Send deadline reminders
FR9	Track document access
FR10	Manage users and permissions
FR11	Maintain audit logs
FR12	Back up and recover documents

B. Non-Functional Requirements

Requirement	Description
Performance	Search results should appear quickly.
Security	Prevent unauthorized access.
Reliability	Protect documents from loss.
Usability	Provide a simple interface.
Scalability	Support more users and documents.
Maintainability	Make the system easy to update.
Availability	Keep the system available during working hours.

3. Identify Actors and Use Cases

Primary Actors

1. Lawyer
2. Legal Assistant
3. Administrator
4. Client

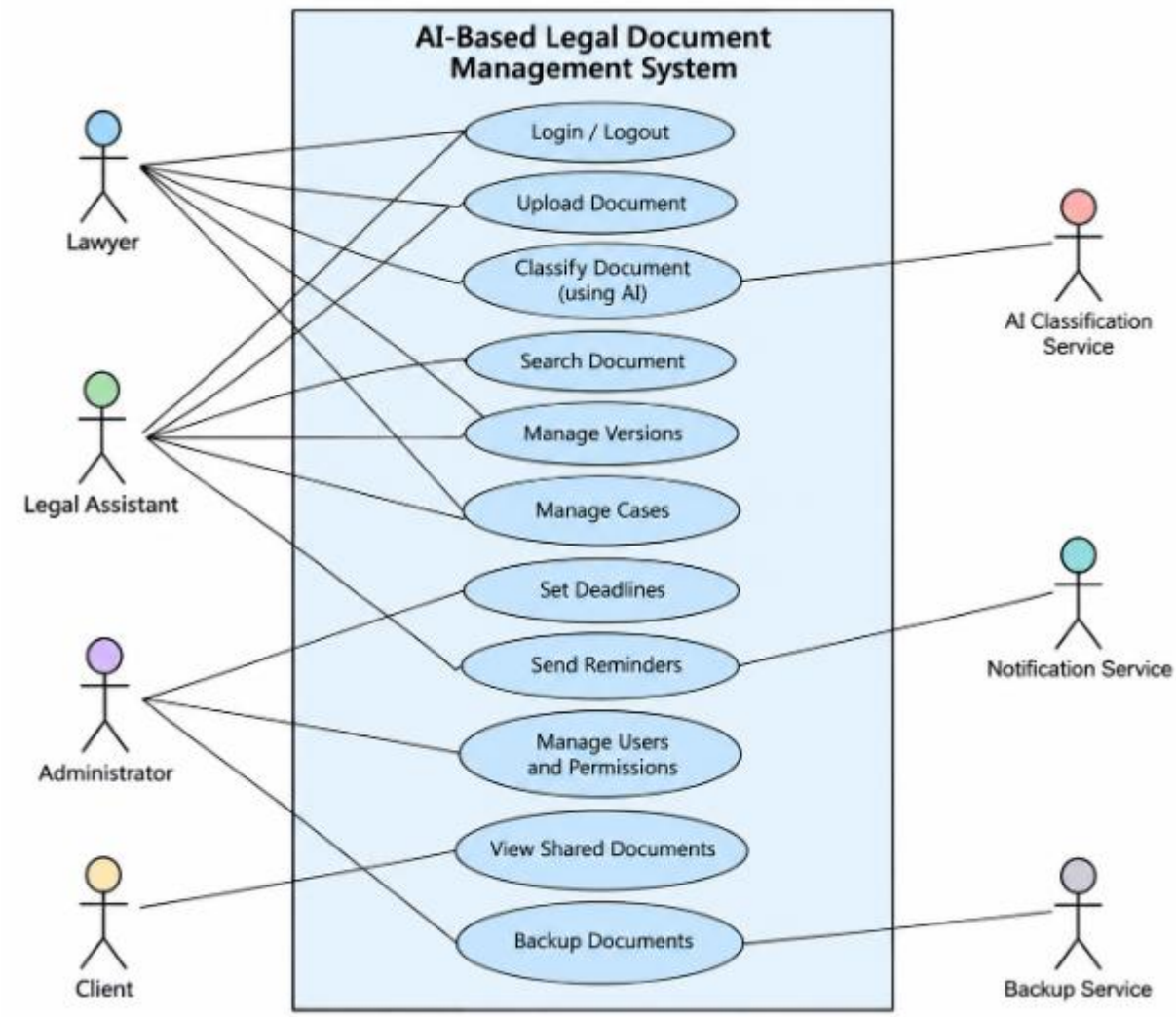
Secondary Actors

1. AI Classification Service
2. Notification Service
3. Backup Service

Use Cases

Actor	**Use Cases**
Lawyer	Upload, search, view, and update documents
Legal Assistant	Organize documents and manage deadlines
Administrator	Manage users and permissions
Client	View shared documents
AI Service	Classify documents
Notification Service	Send deadline reminders
Backup Service	Back up and recover documents

Use Case Diagram



Note:

In the actual UML diagram, draw actors as stick figures, use cases as ovals, and connect each actor to the relevant use cases with association lines.

4. Software Requirement Specification (SRS)

4.1 Introduction

The system uses AI to store, classify, search, and manage legal documents securely.

4.2 Problem Description

Manual document management is time-consuming and can lead to misplaced documents, version confusion, missed deadlines, and security problems.

4.3 Scope of the System

The system allows lawyers and legal assistants to upload, search, classify, and manage documents. It also provides case management, version control, deadline reminders, and access tracking.

4.4 Functional Requirements

- User login and authentication.
- Document upload and storage.
- AI-based document classification.
- Intelligent document search.
- Document version management.
- Case management.
- Deadline reminders.
- Access tracking and audit logs.

4.5 Non-Functional Requirements

- Fast response time.
- Strong security.
- High reliability.
- User-friendly interface.
- Scalability.
- Easy maintenance.
- Good availability.

4.6 Use Case Descriptions

Use Case 1: Upload Document

1. User logs in.
2. User uploads a document.
3. System stores the document.
4. AI classifies the document.
5. System displays a success message.

Use Case 2: Search Document

1. User enters a keyword.
2. System searches the documents.
3. Matching results are displayed.
4. User opens the required document.

Use Case 3: Manage Versions

1. User selects a document.
2. User uploads an updated version.
3. System saves the new version.
4. Previous versions are maintained.

Use Case 4: Deadline Reminder

1. User enters a deadline.
2. System saves the date.
3. System checks approaching deadlines.
4. System sends a reminder.

4.7 Data Requirements

Entity	**Data**
User	User ID, name, email, role

Entity	Data
Document	Document ID, title, category, file path
Version	Version ID, document ID, date
Case	Case ID, case name, status
Deadline	Deadline ID, due date, status
Access Log	User ID, document ID, activity

4.8 External Interface Requirements

- **User Interface:** Login, dashboard, upload, search, and case pages.
- **Software Interface:** Database, AI service, and notification service.
- **Hardware Interface:** Computer and server.
- **Communication Interface:** Secure HTTPS connection.

4.9 System Features

1. Secure document storage.
2. AI-based classification.
3. Intelligent search.
4. Version control.
5. Case management.
6. Deadline reminders.
7. Access tracking.
8. Backup and recovery

5. Identify Assumptions and Constraints

Assumptions

1. Users have valid login credentials.
2. Documents use supported file formats.
3. The AI service is available.
4. Users have appropriate permissions.
5. Deadline details are entered correctly.

Constraints

- **Technical:** AI accuracy depends on document quality.
- **Operational:** Users may require training.
- **Economic:** Storage and maintenance involve costs.
- **Legal:** Confidential documents must be protected.
- **Time:** Development must finish before the deadline.

6. Validate Requirement Completeness and Consistency

Validation	Result
Completeness	All major requirements are included.
Consistency	Requirements work together without conflicts.
Clarity	Requirements are easy to understand.
Redundancy	Unnecessary duplication is avoided.
Feasibility	The system can be developed with available technology.
Testability	Each feature can be tested.
Security	Access controls protect confidential documents.

Suggested Improvements

1. Improve AI classification accuracy.
2. Provide regular backups.
3. Test document security.
4. Improve search speed.
5. Test deadline reminders.

Conclusion

The AI-Based Legal Document Management System securely stores, classifies, and retrieves legal documents. It manages document versions, cases, and deadlines, reducing manual work and improving the efficiency and security of legal teams.